
CS771 Minor Assignment 1

Aiklavyaveer Singh Rana
asrana24@iitk.ac.in

Gade V S S R K Tejas Reddy
tejasreddy24@iitk.ac.in

Karthic N A
akarthic24@iitk.ac.in

Kasavajjala Sai Shreyas
ksshreyas24@iitk.ac.in

Mehta Devarsh Shaunakbhai
devarshms24@iitk.ac.in

Shashwath Ram
shashwathr24@iitk.ac.in

1 Linearly Separable Check

Answer: Yes.

Proof: Let the challenge level be denoted by $L > 0$. From the `genChallengeData` function, we can identify the centers of the generated distributions. Let $v = \sqrt{2}$ and the dynamic shift $c(L) = \sqrt{2} \left(1 + \frac{1}{2L}\right)$. Since $L > 0$, $c(L) > \sqrt{2}$. The function generates data uniformly on the surfaces of circles with a fixed radius $r = 1$. The centers of these circles are:

- Positive Class Circle 1 (P_1): $(-c(L), \sqrt{2})$
- Positive Class Circle 2 (P_2): $(c(L), -\sqrt{2})$
- Negative Class Circle (N): $(-c(L), -\sqrt{2})$

To prove linear separability, we must show that the circles forming the positive class and the circle forming the negative class never intersect or overlap, meaning there is always a strictly positive gap between the two classes. Consider the line connecting the centers of the two positive circles, P_1 and P_2 . The midpoint of this segment is the origin $(0, 0)$, and the slope m of the line is:

$$m = \frac{-\sqrt{2} - \sqrt{2}}{c(L) - (-c(L))} = -\frac{\sqrt{2}}{c(L)}$$

The equation of this connecting line is $y = -\frac{\sqrt{2}}{c(L)}x$, which can be rewritten in standard form as:

$$\sqrt{2}x + c(L)y = 0$$

Next, we calculate the perpendicular distance, D , from the center of the negative circle $N(-c(L), -\sqrt{2})$ to this separating axis:

$$D = \frac{|\sqrt{2}(-c(L)) + c(L)(-\sqrt{2})|}{\sqrt{(\sqrt{2})^2 + c(L)^2}} = \frac{|-2\sqrt{2}c(L)|}{\sqrt{2 + c(L)^2}} = \frac{2\sqrt{2}c(L)}{\sqrt{2 + c(L)^2}}$$

To evaluate if this distance is sufficient for separability, we square it to get D^2 :

$$D^2 = \frac{8c(L)^2}{2 + c(L)^2} = \frac{8}{\frac{2}{c(L)^2} + 1}$$

Now, $c(L) > \sqrt{2}$, which means $c(L)^2 > 2$. Therefore, the fraction $\frac{2}{c(L)^2} < 1$. Substituting this inequality into the denominator yields:

$$\frac{2}{c(L)^2} + 1 < 2$$

$$D^2 > \frac{8}{2} = 4 \implies D > 2$$

The distance from the center of the negative circle to the line connecting the positive circles is strictly greater than 2 for any $L > 0$. Since the data points are located exactly on the circumferences of circles with radius $r = 1$, the positive points extend at most 1 unit towards the negative side of the axis and the negative points extend at most 1 unit towards the positive axis from center N . The total required clearance to prevent overlap is $r + r = 2$. Since $D > 2$, there is always a strictly positive margin of $D - 2 > 0$ separating the positive and negative data points of the two classes. Therefore, the dataset is linearly separable for all valid values of L and n .

2 Experimentation for Small Challenge Levels

In this section, we analyze the performance of a Linear SVM on a Small challenge level ($level < 0.5$). Our objective is to find the smallest values for the hyperparameters myC and max_iter , and the largest value for $myTol$, which yields 100% training accuracy.

2.1 Hyperparameter Tuning and Search Results

Experimental tests were conducted on a fixed challenge level of 0.2 with $n = 500$ points. We systematically varied one hyperparameter at a time while keeping the others fixed to isolate their specific effects on the process.

myC	max_iter	myTol	Acc
1	1000	0.0001	1.0
0.1	1000	0.0001	1.0
0.01	1000	0.0001	1.0
0.0090	1000	0.0001	1.0
0.0080	1000	0.0001	1.0
0.0070	1000	0.0001	1.0
0.0060	1000	0.0001	1.0
0.0040	1000	0.0001	1.0
0.0030	1000	0.0001	1.0
0.0020	1000	0.0001	1.0
0.0010	1000	0.0001	0.9873
0.0011	1000	0.0001	1.0
0.0011	500	0.0001	1.0
0.0011	100	0.0001	1.0
0.0011	50	0.0001	1.0
0.0011	30	0.0001	1.0
0.0011	10	0.0001	1.0
0.0011	5	0.0001	1.0
0.0011	4	0.0001	1.0
0.0011	3*	0.0001	1.0*
0.0011	4	0.001	0.33
0.0011	4	0.01	0.33
0.0011	4	0.1	1
0.0011	4	1	1
0.0011	4	2	1
0.0011	4	5	0.3333
0.0011	4	3	0.3333
0.0011	4	2.5	1.0
0.0011	4	2.9999	1.0
0.0011	3	2.9999	1.0
0.0011	2	2.9999	1.0
0.0011	1*	2.9999	1.0*

* **Note:** An empirical accuracy of 1.0 was technically achieved at that max_iter. However, this triggered a ConvergenceWarning because the solver hit the iteration limit before mathematically converging. Because the easy dataset (level = 0.2) has a massive geometric gap between classes, this early, unoptimized decision boundary was still sufficient to perfectly separate the points.

2.1.1 Optimal Extrema Found:

Based on the results, the extremal hyperparameter configuration for 100% accuracy is:

Hyperparameter	Optimal Value
myC	0.0011
max_iter	2
myTol	2.9999

2.2 Decision Boundary Photo:

The image below is generated with the help of shade2D and plot2D given the extremal hyperparameter values.

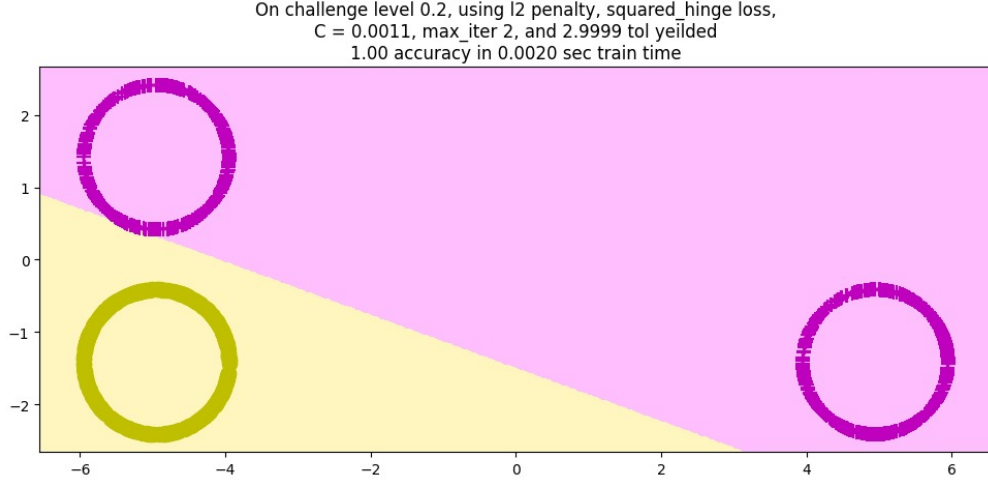


Figure 1: Decision boundary for $level = 0.2$ using optimal extrema.

3 Experimentation for High Challenge Levels

In this section, we analyze the performance of a Linear SVM on a high challenge level ($level > 50$). Our objective is to identify the "Optimal Extrema": the smallest values for the hyperparameters C and max_iter , and the largest value for tol , which yields 100% training accuracy.

3.1 Hyperparameter Tuning and Search Results

Experimental tests were conducted on a fixed challenge level of 55 with $n = 500$ points. We systematically varied one hyperparameter at a time while keeping the others at a high-performing baseline to isolate their specific effects on the process.

Table 1: Comprehensive Hyperparameter Search Results

Table 1: myC			Table 2: max_iter			Table 3: myTol		
C	Acc.	Time	iter	Acc.	Time	tol	Acc.	Time
1e-05	77.80%	.0021	1	93.60%	.0016	1e-06	100.0%	.0027
0.0001	83.33%	.0020	2	93.40%	.0020	1e-05	100.0%	.0026
0.001	96.33%	.0022	5	96.93%	.0024	0.0001	100.0%	.0029
0.005	93.87%	.0022	6	97.47%	.0025	0.001	99.60%	.0027
0.01	94.00%	.0023	7	97.93%	.0024	0.002	99.60%	.0027
0.05	95.27%	.0023	8	98.47%	.0025	0.003	98.47%	.0024
0.1	95.93%	.0024	9	99.07%	.0028	0.004	98.47%	.0024
1.0	97.27%	.0024	10	99.60%	.0028	0.005	98.47%	.0024
10.0	98.47%	.0026	11	99.60%	.0026	0.006	98.47%	.0024
20.0	98.87%	.0029	12	100.0%	.0028	0.007	98.47%	.0024
30.0	99.00%	.0026	13	100.0%	.0026	0.008	98.47%	.0026
40.0	99.33%	.0027	14	100.0%	.0027	0.009	98.47%	.0026
50.0	99.40%	.0026	15	100.0%	.0027	0.01	98.47%	.0025
60.0	99.53%	.0028	20	100.0%	.0026	0.1	95.87%	.0023
70.0	99.60%	.0027	50	100.0%	.0027	0.5	93.60%	.0019
80.0	99.80%	.0027	100	100.0%	.0028	1.0	93.60%	.0020
90.0	99.87%	.0029	200	100.0%	.0026	5.0	33.33%	.0018
100.0	100.0%	.0031	500	100.0%	.0027	10.0	33.33%	.0017
1000.0	100.0%	.0022	1000	100.0%	.0026	50.0	33.33%	.0017
			5000	100.0%	.0025			

3.1.1 Optimal Extrema Found:

Based on the results, the extremal hyperparameter configuration for 100% accuracy is:

Hyperparameter	Optimal Value
myC	100.0
max_iter	12
myTol	0.0001

3.2 Decision Boundary Photo:

The image below is generated with the help of shade2D and plot2D given the extremal hyperparameter values.

On challenge level 55, using l2 penalty, squared_hinge loss,
C = 100.0, max_iter 12, and 0.0001 tol yielded
1.00 accuracy in 0.0036 sec train time

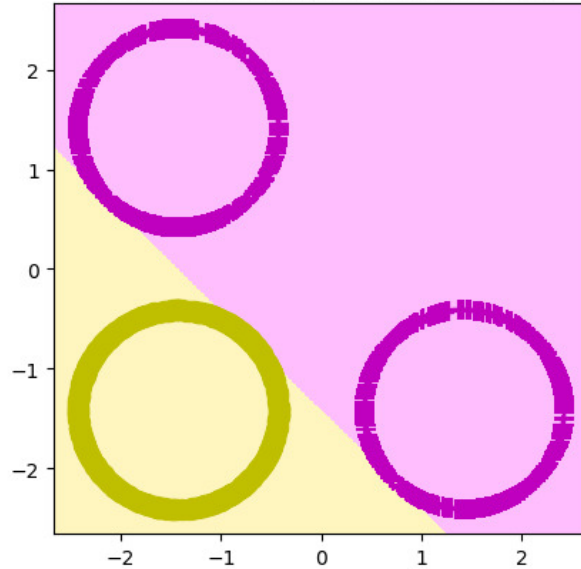


Figure 2: Decision boundary for $level = 55$ using optimal extrema.

4 Hyperparameter Sensitivity Analysis

4.1 Experimental Setup

In this task, we tested how different settings (hyperparameters) change the results of an SVM model. We used a medium difficulty dataset ($myLevel = 5.0$) with $n = 1000$ points. We tested three settings:

- **C**: How much the model avoids mistakes.
- **max_iter**: How many times the model tries to find the best line.
- **tol**: When the model should stop working.

For each setting, we changed one value while keeping the others the same.

4.2 Effect of Regularization (C)

The value C tells the model how much to care about misclassified points. As shown in Table 2, when C is very small (0.0001), the accuracy is lower (87.27%). This is because the model is too relaxed. When C is 0.1 or higher, the accuracy stays at 100%. The time taken to train does not change much as C increases.

Table 2: How C changes Accuracy and Time

Value of C	Accuracy (%)	Train Time (s)
0.0001	87.27%	0.003396
0.001	96.90%	0.002216
0.01	96.50%	0.002353
0.1	100.00%	0.002036
1.0	100.00%	0.001721
10	100.00%	0.001654
100	100.00%	0.002175
1000	100.00%	0.001853
10000	100.00%	0.002075
100000	100.00%	0.002235

4.3 Effect of Maximum Iterations (max_iter)

The value max_iter is the limit for how many steps the model can take. For this data, the model reached 100% accuracy very quickly, even with only 10 steps. However, as shown in Table 3, if we let the model take up to 10,000 steps, the training time increases to 0.004056 seconds because the computer has to do more checks.

Table 3: How max_iter changes Accuracy and Time

max_iter	Accuracy (%)	Train Time (s)
10	100.00%	0.002620
50	100.00%	0.002259
100	100.00%	0.002137
250	100.00%	0.002500
500	100.00%	0.002555
1000	100.00%	0.002491
2000	100.00%	0.002239
4000	100.00%	0.002820
7000	100.00%	0.003558
10000	100.00%	0.004056

4.4 Effect of Tolerance (tol)

Tolerance (tol) tells the model to stop once it stops improving by a certain amount. The accuracy stayed at 100% for all tested values. As seen in Table 4, the fastest training time was when tol was around 0.001. When tol is very small ($1e - 07$), it takes a bit more time because the model is trying to be more precise.

Table 4: How tol changes Accuracy and Time

Value of tol	Accuracy (%)	Train Time (s)
0.1	100.00%	0.002420
0.05	100.00%	0.002190
0.01	100.00%	0.002202
0.005	100.00%	0.001681
0.001	100.00%	0.001604
0.0005	100.00%	0.001809
0.0001	100.00%	0.001683
1e-05	100.00%	0.001619
1e-06	100.00%	0.002199
1e-07	100.00%	0.002506

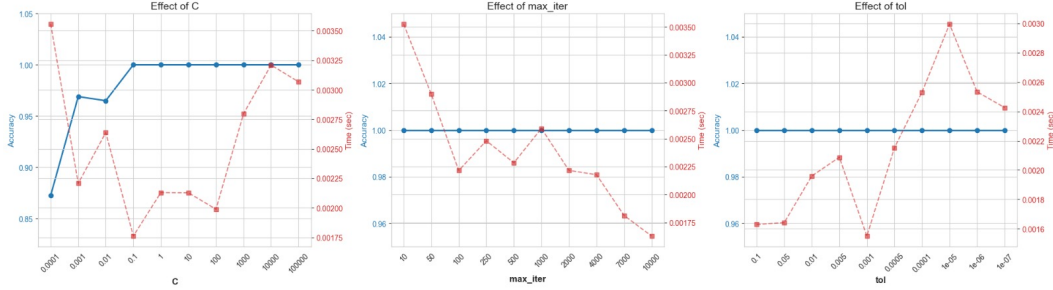


Figure 3: Hyperparameter Sensitivity Analysis (level 5.0)

4.5 Key insights:

The SVM finds the perfect boundary very quickly, reaching almost 100% accuracy in just 10 iterations when $C \geq 0.1$. As the model gets good accuracy so quickly, extreme settings only impose a higher computational overhead. The model’s answer is most sensitive to **C**; **max_iter** and **tol** only seem to influence efficiency rather than the answer. So in these cases, tuning these parameters simply teaches the model exactly when to stop.

5 Effect of Number of Training Points on Training Time and Accuracy

The following results have been evaluated for $\text{myLevel} = 60$, using LinearSVC with ℓ_2 penalty, squared hinge loss, $C = 0.1$, $\text{max_iter} = 100$, and $\text{tol} = 10^{-4}$. For each value of n , training was repeated 5 times and the average training time was recorded. We explored n ranging from 10 to 15,000 in 50 discrete, linearly spaced steps.

5.1 Effect on Training Time

From the left plot in Figure 4, the training time grows roughly **linearly** with the number of training points n . The training time increases from approximately 0.0026 seconds at $n = 10$ to roughly 0.044–0.058 seconds at $n = 15,000$. The growth is largely linear throughout this range, although some variance is visible for larger n . This variance likely arises because the solver takes a varying number of gradient descent iterations depending on the difficulty of the optimization landscape as the dataset size grows.

5.2 Effect on Accuracy

From the right plot in Figure 4, we observe **diminishing returns** in training accuracy as n increases. The classifier starts at approximately 93.3% accuracy for $n = 10$, rises steeply to around 97% by $n \approx 1,000$, and thereafter gradually improves, plateauing at approximately 97.8–98.0% for

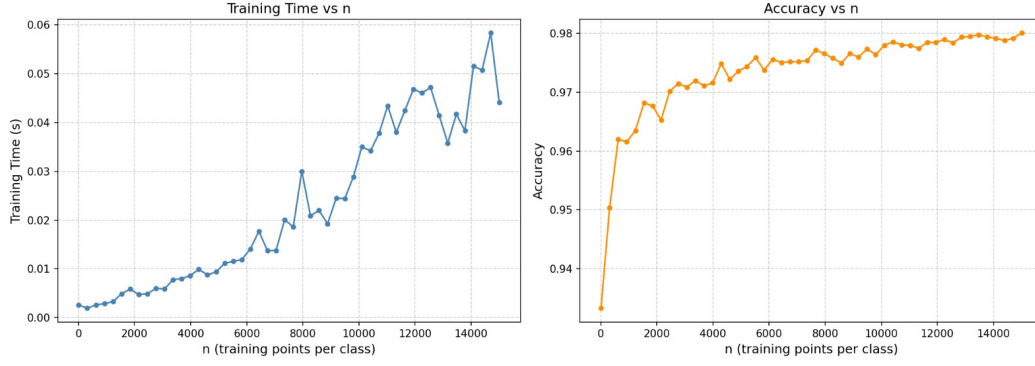


Figure 4: Training time (left) and training accuracy (right) as a function of the number of training points per class n , at challenge level = 60.

$n \gtrsim 5,000$. Beyond this point, adding more training data contributes only marginally to accuracy. This indicates that the model effectively learns the underlying data distribution with around 5,000 points per class; additional data beyond this threshold yields negligible improvement, which is consistent with the high separability of the data at level = 60.